

Uso de métricas em testes de integração para otimização do tempo de validação de mudanças em pipelines DevOps.

Gabriel Alves Pereira, Matheus Emanuel Lopes de Souza, Sara Cristina Correia da Silva

¹Universidade Federal de Rondonópolis (UFR)

Abstract. *The use of metrics in integration testing is an important strategy for optimizing change validation in DevOps pipelines. This study aimed to conduct a literature analysis and evaluate pipelines in a simulated fintech application representing a financial system. Additionally, testing architectures, automation tools, and metrics related to pipeline performance were analyzed. The research was carried out through a literature mapping based on scientific articles. Searches were performed on Google Scholar, considering review papers published between 2022 and 2025. After applying the selection criteria, ten studies were included in the final synthesis. The results highlight the importance of metrics for identifying bottlenecks and improving the efficiency of the software development process.*

Resumo. *O uso de métricas em testes de integração é uma estratégia importante para otimizar a validação de mudanças em pipelines DevOps. Este trabalho teve como objetivo análise literaria e testes de pipelines em uma aplicação fintech simulada de um sistema financeiro. Este trabalho teve como objetivo analisar arquiteturas de teste, ferramentas de automação e métricas relacionadas ao desempenho dos pipelines. A pesquisa foi conduzida por meio de um mapeamento da literatura baseada em artigos científicos. As buscas foram realizadas no Google Scholar, considerando artigos de revisão publicados entre 2022 e 2025. Após a aplicação dos critérios de seleção, 10 estudos foram incluídos na síntese final. Os resultados evidenciam a importância das métricas para identificar gargalos e melhorar a eficiência do processo de desenvolvimento de software.*

1. Introdução

As métricas modernas de DevOps servem como um mecanismo fundamental para avaliar o desempenho da entrega de software e a saúde do sistema. Entre os frameworks mais adotados nessa área está o modelo de métricas DevOps Research and Assessment (DORA), que se tornou um parâmetro global (DARAOJIMBA et al., 2024, p. 102). Além de fornecer indicadores quantitativos para a avaliação da maturidade DevOps, a aplicação das métricas DORA tem sido associada a ganhos de previsibilidade e estabilidade nas liberações de software em diferentes contextos organizacionais (TAIWO et al., 2021).

Ferramentas como Prometheus e Grafana tornaram-se essenciais para a coleta, armazenamento e visualização de métricas de desempenho em ambientes modernos de entrega de software. Essas ferramentas permitem a análise de séries temporais e proporcionam elevados níveis de observabilidade sobre o comportamento dos sistemas, fornecendo informações relevantes para o monitoramento e a melhoria contínua dos processos de desenvolvimento (DARAOJIMBA et al., 2024).

O objetivo geral deste artigo é investigar como a utilização de testes de integração, ferramentas de automação e métricas de desempenho em ambientes DevOps de fintechs pode contribuir para a otimização do tempo de validação de mudanças e para o aumento da confiabilidade dos pipelines de entrega contínua.

Nesse contexto, busca-se responder às seguintes questões de pesquisa: Quais são os principais gargalos de produtividade e desempenho associados às métricas empregadas em testes de integração durante o desenvolvimento de software? E quais contramedidas podem ser adotadas para reduzir seus impactos na produtividade e, consequentemente, na qualidade e no tempo de entrega do produto final? (RQ1)

Além disso, pretende-se compreender de que forma a utilização de métricas pode tornar o processo de entrega de software mais eficiente e confiável, fornecendo subsídios para a tomada de decisões baseada em dados e promovendo a melhoria contínua dos processos (RQ2).

Por fim, busca-se analisar a importância da identificação e da adoção de ferramentas de testes, monitoramento, segurança e conformidade mais adequadas ao contexto do mercado financeiro (RQ3).

2. Metodologia

2.1. Tipo de Pesquisa

Esta pesquisa caracteriza-se como um Mapeamento Sistemático da Literatura (MSL), com o objetivo de identificar e analisar os estudos relacionados ao uso de testes de integração, métricas de desempenho e práticas DevOps em ambientes de desenvolvimento de software, com ênfase em aplicações do setor financeiro. Para isso, foi elaborada uma string de busca baseada nos principais termos associados ao tema investigado. A string foi aplicada no Google Acadêmico, permitindo a identificação e a seleção dos trabalhos mais relevantes para o tema escolhido. Adicionalmente, foi conduzida uma experimentação com o propósito de simular um ambiente de desenvolvimento baseado em práticas DevOps, possibilitando a coleta e a análise de métricas de desempenho por meio das ferramentas Prometheus e Grafana. Essa etapa experimental teve como finalidade complementar os resultados obtidos na literatura, fornecendo evidências práticas sobre o comportamento e o monitoramento dos pipelines de integração e entrega contínua.

2.2. Fontes de informação

A construção da string foi baseada nos principais conceitos relacionados ao tema da pesquisa, abrangendo o domínio de aplicação (fintechs), as práticas de desenvolvimento (DevOps e entrega contínua) e as métricas de desempenho utilizadas em pipelines de integração e entrega contínua.

2.3. Seleção dos Estudos

Após a aplicação da string de busca no Google Acadêmico, os estudos recuperados foram submetidos a um processo de seleção baseado na análise dos títulos, resumos e, quando necessário, do conteúdo completo dos trabalhos. Foram considerados para inclusão os estudos que apresentavam relação direta com o tema da pesquisa. Esse processo teve como objetivo garantir a relevância e a qualidade dos estudos considerados na análise do mapeamento sistemático da literatura.

Tabela 1. String de busca utilizada no estudo

Strings
("fintech"OR "financial technology"OR "digital banking") AND ("DevOps"OR "CI/CD"OR "continuous delivery") AND ("DORA metrics"OR "deployment frequency"OR "change failure rate"OR "pipeline metrics")

2.4. Extração e Agrupamento das Informações

Após a seleção dos estudos, foram extraídas informações relevantes, como título, autores, ano de publicação, ferramentas empregadas e temática abordada. Esses dados foram organizados em tabelas e posteriormente correlacionados com as informações obtidas na etapa experimental. Ao final, realizou-se uma análise comparativa entre os resultados encontrados na literatura e as métricas coletadas durante a experimentação, com o objetivo de identificar convergências, diferenças e evidências práticas relacionadas ao tema investigado.

3. Resultados

3.1. Caracterização dos Estudos Selecionados

Foram analisados 10 trabalhos, selecionados com critérios alinhados a PRISMA e às questões de pesquisa (RQ-1 a RQ-3). A maioria é revisão sistemática ou síntese aplicada; quatro artigos tratam diretamente de fintech ou software financeiro.

Tabela 2. Caracterização dos estudos selecionados

#	Autor(es)	Ano	Tema central
P1	Nagraj	2022	GitOps, CD e DevSecOps em fintech
P2	Daraojimba et al.	2024	Métricas DevOps/SRE e DORA
P3	Oyewole et al.	2023	Maturidade DevOps
P4	Sakinala	2025	IA aplicada a DevOps
P5	Jyoti et al.	2024	Automação de testes em camadas
P6	Adewusi et al.	2022	Métricas e OKRs
P7	Al-Habib, Sullaiman	2024	Microserviços, containers e observabilidade
P8	Argaz et al.	2023	Decomposição monólito - microserviços
P9	Sain et al.	2025	Decisão em métodos ágeis
P10	Onyeagusi, Onyeagusi	2025	DevOps em software enterprise/fintech

3.2. Panorama Temático

A análise dos estudos selecionados permitiu a organização dos resultados em cinco eixos temáticos principais. O primeiro eixo, Métricas e Desempenho (P2, P3, P6 e P7), concentra-se na utilização de indicadores como *lead time*, taxa de falha nas mudanças (*change failure rate*) e tempo médio para recuperação (*MTTR*), destacando as métricas DORA como referência para avaliação da maturidade e eficiência dos processos DevOps.

O segundo eixo, Pipelines e Entrega Contínua (P1, P3 e P10), aborda a adoção de práticas como GitOps e Continuous Delivery (CD), evidenciando seus impactos positivos no aumento da frequência de releases e na agilidade dos processos em ambientes fintech.

Em Qualidade e Testes (P1 e P5), os estudos enfatizam a importância da automação de testes em diferentes camadas da aplicação, indicando que testes de integração e de APIs apresentam maior efetividade quando comparados à utilização exclusiva de testes de interface.

O eixo de Infraestrutura e Observabilidade (P7 e P8) destaca a adoção de arquiteturas baseadas em microserviços e containers, bem como o emprego de ferramentas de monitoramento e rastreamento, como Docker, Prometheus, Grafana e Jaeger, para aumentar a visibilidade e a confiabilidade dos sistemas.

Por fim, o eixo Decisão e Tendências (P4, P9 e P10) reúne trabalhos relacionados à aplicação de inteligência artificial em processos DevOps e à tomada de decisão em ambientes ágeis. Observou-se que ainda existem poucas evidências na literatura sobre mecanismos de apoio à decisão voltados para testes e processos de liberação de software, enquanto a incorporação de técnicas de inteligência artificial é apontada como uma tendência promissora para a evolução das práticas DevOps.

De maneira geral, verificou-se uma lacuna na literatura quanto à comparação experimental de diferentes toolchains DevOps aplicadas a um mesmo sistema sob condições controladas. A maioria dos estudos analisa ferramentas e práticas de forma isolada, motivando a realização do experimento desenvolvido neste trabalho, que busca comparar abordagens distintas em um ambiente controlado e avaliar seus impactos sobre métricas de desempenho e confiabilidade.

3.3. Práticas DevOps Identificadas

Tabela 3. Práticas DevOps identificadas e implementadas

Prática	O que a literatura recomenda	O que implementamos
CI/CD	Validar cada mudança antes do deploy	Pipelines A e B automatizadas
GitOps	Git como fonte da verdade	Workflows versionados e tri- lha de auditoria
Quality Gates	Lint, tipos e análise estática	ESLint, Prettier, TypeScript e Sonar (opcional)
Testes em camadas	Unitário - Integração - E2E - Performance	Vitest/Jest, Testcontainers/Compose, Playwright/Cypress e k6/Artillery
Observabilidade	Métricas e traces	Jaeger (Pipeline A); Prometheus/Grafana (Pipeline B)
Métricas DORA	Tempo, falhas e frequência	Exportação em JSON por execução e tempo por etapa
Cenários controlados	Validação reprodutível	Cenários C0–C4 por meio da variável EXPERIMENT_SCENARIO

3.4. Ferramentas DevOps Mapeadas

Tabela 4. Ferramentas DevOps mapeadas por pipeline

Etapa do pipeline	Pipeline A	Pipeline B
CI	GitHub Actions (job único)	GitLab CI (multi-stage)
Unitários	Vitest	Jest
Integração	Testcontainers + PostgreSQL	Docker Compose + PostgreSQL
E2E	Playwright	Cypress
Performance	k6	Artillery
Observabilidade	OpenTelemetry + Jaeger	Prometheus + Grafana
Deploy	Vercel Preview (opcional)	Docker Compose (opcional)

A Tabela 4 apresenta as ferramentas empregadas em cada etapa dos pipelines avaliados. Observa-se que o Pipeline A possui uma estrutura mais simplificada, utilizando um único job de integração contínua e ferramentas voltadas para rastreamento distribuído por meio do OpenTelemetry e Jaeger. Em contraste, o Pipeline B apresenta uma configuração mais elaborada, composta por múltiplos estágios e uma infraestrutura de observabilidade baseada em Prometheus e Grafana. De acordo com o modelo de maturidade DevOps proposto por Oyewole et al. (2023), o Pipeline A pode ser associado ao nível *Managed*, enquanto o Pipeline B se aproxima do nível *Defined*, caracterizado pela presença de maior encadeamento de etapas, observabilidade em ambiente próprio e maior controle sobre os processos de integração e entrega contínua.

3.5. Pipeline DevOps Conceitual

Fluxo adotado, alinhado à literatura e ao protótipo:

Código → Validação → Build → Testes unitários → Integração → E2E → Performance → Quality gate → Deploy → Monitoramento

Tabela 5. Cenários controlados e pontos de falha esperados

Cenário	Defeito simulado	Onde falha
C0	Nenhum	Pipeline verde
C1	Erro de lint	Validação
C2	Bug de saldo em transferência	Integração
C3	Erro de build	Build
C4	Regressão de latência	Performance

A Tabela 5 apresenta os cenários controlados utilizados durante a etapa experimental e os respectivos pontos de falha esperados em cada pipeline. Os defeitos foram introduzidos de forma controlada por meio de uma variável de ambiente, permitindo a

reprodução dos experimentos sem a necessidade de alterações permanentes no código-fonte da aplicação. Essa abordagem possibilitou a avaliação do comportamento dos pipelines diante de diferentes tipos de falhas, contribuindo para a análise da capacidade de detecção de erros e do impacto desses cenários sobre as métricas de desempenho e confiabilidade.

4. Discursão

4.1. Resposta às Questões de Pesquisa

RQ-1 (métricas e desempenho): A literatura (P2, P3, P7) aponta métricas DORA como referência. No experimento, coletamos lead time (72–164 s), taxa de falha (100% em C2, quando desejável) e tempo por etapa. Conclusão: métricas DORA são aplicáveis e automatizáveis em pipelines fintech simulados.

RQ-2 (tempo de validação e decisão): P6 e P9 alertam que métricas isoladas não orientam decisões em teste/release. O experimento mostrou que A e B têm tempo equivalente (2 s de diferença) e mesma detecção em C2. Conclusão: a escolha de toolchain deve considerar infraestrutura (cloud vs on-premise), não só duração — omitir integração reduz tempo, mas aumenta risco de falso negativo.

RQ-3 (testes e confiabilidade): P5 defende testes em camadas; P1 exige validação rigorosa em fintech. No cenário C2, unitários passaram e integração falhou em 100% dos casos (A e B). Conclusão: regras financeiras (saldo, transferência, auditoria) exigem testes de integração com banco real.

Tabela 6. Síntese comparativa entre os pipelines

Critério	Pipeline A	Pipeline B
Tempo (C0, local)	~74 s	~72 s
Deteção C2	100%	100%
Etapas de deteção	Integração	Integração
Perfil	Cloud, feedback rápido	On-premise, multi-stage
CI completo (E2E + k6)	164 s (GitHub Actions)	Pendente no GitLab

Tabela 7. Resultado das hipóteses avaliadas

Hipótese	Resultado
H1 — B detecta mais, porém é mais lento	Parcialmente refutada: ambos os pipelines apresentaram a mesma capacidade de detecção, além de tempos de execução equivalentes.
H2 — C2 só falha em integração/E2E	Confirmada. O defeito inserido no cenário C2 foi detectado apenas nas etapas de integração, corroborando a importância dos testes em camadas.
H3 — A mais rápido; B melhor em infraestrutura	Parcialmente confirmada: os tempos observados foram semelhantes, enquanto o Pipeline B apresentou características mais adequadas para ambientes on-premise e arquiteturas com múltiplos estágios.